VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA", BELAGAVI - 590 018

NEURAL NETWORKS AND DEEP LEARNING LABORATORY

(21AIL75) REPORT

Submitted by

Shreya **4SF21AD050**

In partial fulfilment of the requirements for the VII semester

of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

at

SAHYADRI

COLLEGE OF ENGINEERING & MANAGEMENT

An Autonomous Institution

Adyar, Mangaluru - 575 007

Academic Year: 2024 - 25

SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

CERTIFICATE

This is to certify that Ms. Shreya bearing USN: 4SF21AD050 has satisfactorily completed the course of experiments in Neural Networks and Deep Learning Laboratory (21AIL75) in partial fulfilment of the requirements of VII semester of Bachelor of Engineering Degree Course in Artificial Intelligence & Data Science as prescribed by the Visvesvaraya Technological University during the year 2024-25.

Date:

Internal Assessment Marks

Faculty In charge

Experiment Details

Experiment No.	Experiment Name	Date of Execution	Page No.	Marks (10)
1	Activation Functions – sigmoid, tanh, ReLU and softmax	24/09/2024	1	
2	a. Single unit Perceptron b. AND, OR, XOR for single unit perception	01/10/2024	4	
3	Deep Feed-forward Neural Network	08/10/2024	8	
4	CNN to classify multi-category image dataset	15/10/2024	10	
5	Image classification model using Deep feed-forward neural network	07/11/2024	12	
6	Bi-Directional LSTM for Sentiment analysis	21/11/2024	14	
7	Standard VGG16 and 19 CNN architecture model to classify multicategory image dataset	05/12/2024	16	

Lab Experiment - 1

1. **Write a program to demonstrate the working of different activation functions like sigmoid, tanh, ReLU & softmax to train a neural network.**

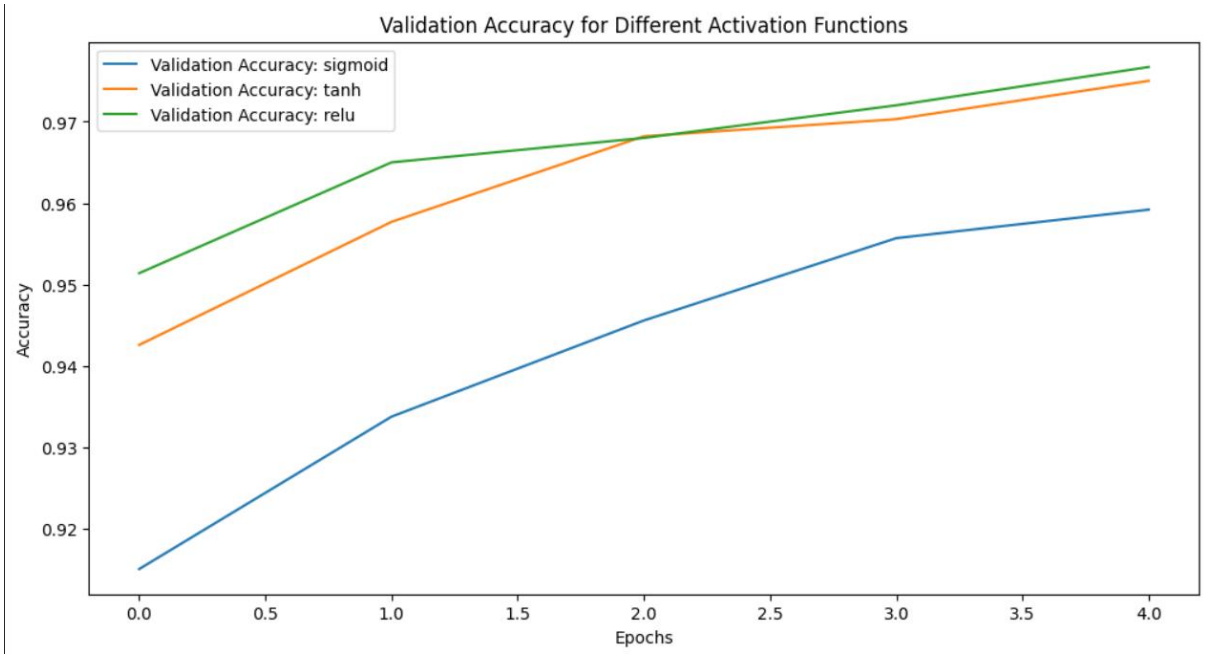
Program:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Load MNIST dataset (handwritten digits 0-9)
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Preprocess the data
X_train = X_train.reshape(X_train.shape[0], 28 * 28).astype('float32') / 255  # Flattening and normalization
X_test = X_test.reshape(X_test.shape[0], 28 * 28).astype('float32') / 255

# One-hot encoding of the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Define the neural network model
def build_model(activation_hidden, activation_output):
    model = Sequential()
    # Input layer and first hidden layer
    model.add(Dense(128, input_shape=(28*28,)))  # 128 neurons in the hidden layer
```

```
    model.add(Activation(activation_hidden))  # Applying the chosen activation function

    # Second hidden layer
    model.add(Dense(64))  # 64 neurons in second hidden layer
    model.add(Activation(activation_hidden))
    # Output layer
    model.add(Dense(10))  # 10 output neurons (since we have 10 classes, digits 0-9)
     model.add(Activation(activation_output))  # Activation function for output (softmax for classification)
    # Compile the model
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

# List of activation functions to use for the hidden layers
activation_functions_hidden = ['sigmoid', 'tanh', 'relu']

# Output layer activation is softmax for all cases since it's a classification task
activation_output = 'softmax'

# Dictionary to store training histories for each activation function
history_dict = {}

# Train and evaluate models with different activation functions for hidden layers
for activation_hidden in activation_functions_hidden:
    print(f"\nTraining model with hidden layer activation: {activation_hidden}")

    # Build the model
    model = build_model(activation_hidden, activation_output)

    # Train the model
     history = model.fit(X_train, y_train, epochs=5, batch_size=128, validation_data=(X_test, y_test), verbose=2)
```

```
    history_dict[activation_hidden] = history
# Plotting the training and validation accuracy for comparison
plt.figure(figsize=(12, 6))
for activation_hidden in activation_functions_hidden:
        plt.plot(history_dict[activation_hidden].history['val_accuracy'], label=f'Validation Accuracy: {activation_hidden}')
plt.title('Validation Accuracy for Different Activation Functions')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Output:



Lab Experiment - 2

2.

a. **Design a single unit perceptron for classification og linearly separable binary dataset without using pre-defined models. Use the perceptron from sklearn.**
b. **Identify the problems in single unit perceptron using AND, OR, XOR data and analyze the results.**

Program:

A.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.linear_model import Perceptron

# Create a linearly separable dataset
X, y = make_blobs(n_samples=100, centers=2, cluster_std=1.0, random_state=42)
# Create the Perceptron model
perceptron = Perceptron(max_iter=1000, eta0=0.01, random_state=42
# Fit the model
perceptron.fit(X, y)
# Function to plot dataset and decision boundary in the same graph
def plot_data_and_decision_boundary(model, X, y):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                         np.arange(y_min, y_max, 0.01))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
```

```
    # Plot the decision boundary
    plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
    # Plot the dataset points
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', edgecolors='k')
    # Add title and labels
    plt.title('Linearly Separable Dataset and Perceptron Decision Boundary')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.show()

# Plot the data and decision boundary in a single graph
plot_data_and_decision_boundary(perceptron, X, y)
```

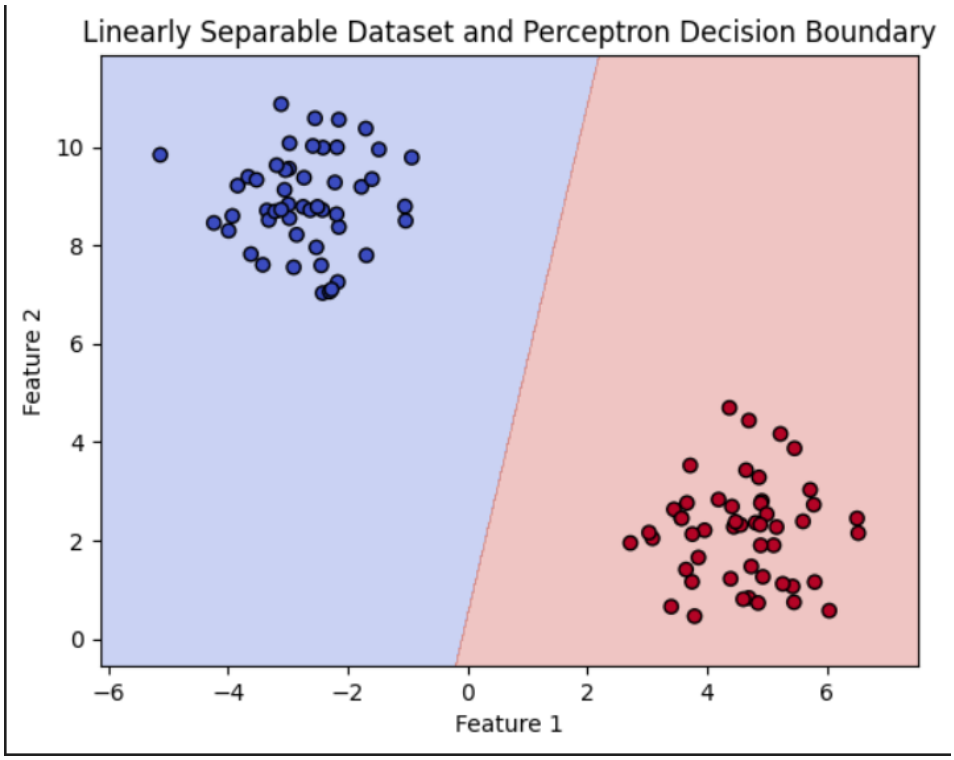
B.

```
from sklearn.linear_model import Perceptron
import numpy as np
import matplotlib.pyplot as plt

# Define the datasets
datasets = {
    "AND": (np.array([[0, 0], [0, 1], [1, 0], [1, 1]]), np.array([0, 0, 0, 1])),
    "OR": (np.array([[0, 0], [0, 1], [1, 0], [1, 1]]), np.array([0, 1, 1, 1])),
    "XOR": (np.array([[0, 0], [0, 1], [1, 0], [1, 1]]), np.array([0, 1, 1, 0]))
}

# Function to train and evaluate the perceptron
def train_and_evaluate(X, y, title):
    perceptron = Perceptron(max_iter=1000, eta0=1, random_state=0).fit(X, y)
    predictions = perceptron.predict(X)

    # Plot data and predictions
```

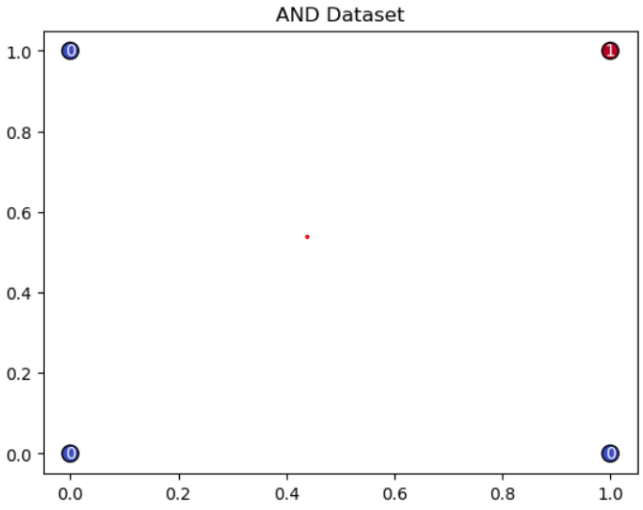
```
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', edgecolor='k', s=100)
    for i, pred in enumerate(predictions):
        plt.text(X[i, 0], X[i, 1], str(pred), color='white', ha='center', va='center')
    plt.title(f'{title} Dataset')
    plt.show()

    print(f'{title} Accuracy: {np.mean(predictions == y) * 100:.2f}%\n')

# Evaluate datasets
for name, (X, y) in datasets.items():
    train_and_evaluate(X, y, name)
```

Output:

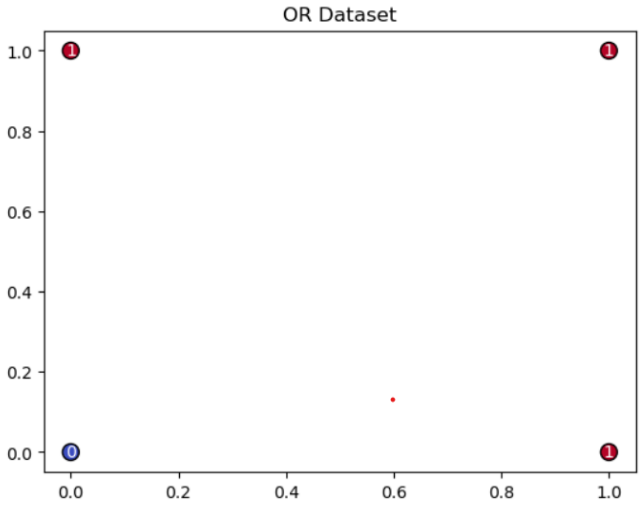
A.



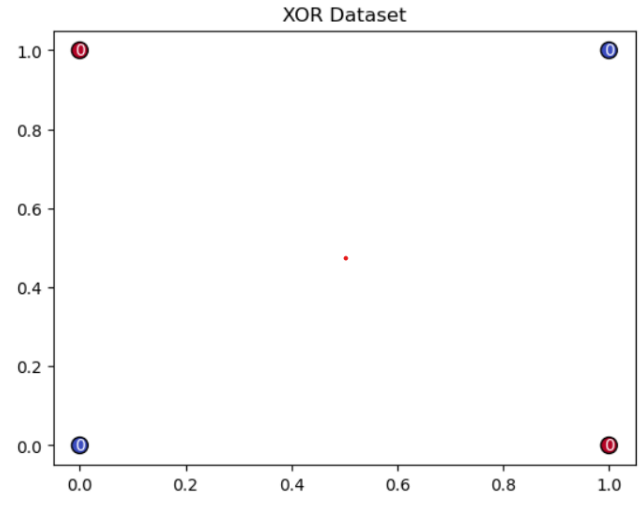
B.



AND Accuracy: 100.00%



OR Accuracy: 100.00%



XOR Accuracy: 50.00%

Lab Experiment - 3

3. Build a deep feed-forward Artificial Neural Network by implementing the back propagation algorithm and test the same using appropriate datasets. Use the number of hidden layers greater than or equal to 4.

Program:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

# Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# One-hot encode the labels
y = to_categorical(y)

# Standardize the features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build the neural network model with four hidden layers
model = Sequential([
```

```
    Dense(64, activation='relu', input_shape=(X.shape[1],)),  # Input layer + first hidden layer
    Dense(32, activation='relu'),  # Second hidden layer
    Dense(16, activation='relu'),  # Third hidden layer
    Dense(8, activation='relu'),   # Fourth hidden layer
    Dense(y.shape[1], activation='softmax')  # Output layer
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=8, verbose=1)

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

Output:

```
15/15 [==============================] - 2s 4ms/step - loss: 1.0203 - accuracy: 0.7583
Epoch 2/10
15/15 [==============================] - 0s 3ms/step - loss: 0.8941 - accuracy: 0.8250
Epoch 3/10
15/15 [==============================] - 0s 2ms/step - loss: 0.7531 - accuracy: 0.8417
Epoch 4/10
15/15 [==============================] - 0s 2ms/step - loss: 0.6182 - accuracy: 0.8500
Epoch 5/10
15/15 [==============================] - 0s 2ms/step - loss: 0.5174 - accuracy: 0.8750
Epoch 6/10
15/15 [==============================] - 0s 2ms/step - loss: 0.4316 - accuracy: 0.9083
Epoch 7/10
15/15 [==============================] - 0s 2ms/step - loss: 0.3576 - accuracy: 0.9333
Epoch 8/10
...
15/15 [==============================] - 0s 2ms/step - loss: 0.2475 - accuracy: 0.9083
Epoch 10/10
15/15 [==============================] - 0s 2ms/step - loss: 0.2017 - accuracy: 0.9417
Test Accuracy: 100.00%
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Lab Experiment - 4

4. **Design and implement a CNN model with 4+ layers of convolution to classify multicategory image datasets. Use the concept of regularization and dropout while designing CNN model. Use the fashion MNIST datset. Record the training accuracy corresponding to the following architecture:**
    a. **Base model**
    b. **Model with L1 Regularization**
    c. **Model with L2 Regularization**
    d. **Model with Dropout**

Program:

```
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.datasets import fashion_mnist
# Load and preprocess the dataset
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
x_train = x_train[..., None]  # Add channel dimension
x_test = x_test[..., None]
y_train, y_test = tf.keras.utils.to_categorical(y_train, 10), tf.keras.utils.to_categorical(y_test, 10)
# Function to create the model with optional regularization or dropout
def create_model(regularizer=None, dropout_rate=None):
    model = models.Sequential([
        layers.Input(shape=(28, 28, 1)),
        layers.Conv2D(32, (3, 3), activation='relu'),
        layers.MaxPooling2D(),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D(),
        layers.Flatten(),
        layers.Dense(128, activation='relu', kernel_regularizer=regularizer),
```

```
        layers.Dropout(dropout_rate) if dropout_rate else layers.Dense(128, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
# List of configurations for model creation
configurations = [
    ("Base Model", None, None),
    ("Model with L1 Regularization", regularizers.l1(1e-4), None),
    ("Model with L2 Regularization", regularizers.l2(1e-4), None),
    ("Model with Dropout", None, 0.5)
]
# Train and evaluate each model configuration
for name, regularizer, dropout_rate in configurations:
    print(f"\nTraining {name}...")
    model = create_model(regularizer, dropout_rate)
    model.fit(x_train, y_train, epochs=5, batch_size=32, verbose=2)
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    print(f"{name} Test Accuracy: {test_acc:.4f}")
```

Output:

```
Epoch 1/5
WARNING:tensorflow:From c:\Users\Shreya\AppData\Local\Programs\Python\Python311\Lib\si

WARNING:tensorflow:From c:\Users\Shreya\AppData\Local\Programs\Python\Python311\Lib\si

1875/1875 - 22s - loss: 0.4600 - accuracy: 0.8310 - 22s/epoch - 12ms/step
Epoch 2/5
1875/1875 - 18s - loss: 0.2992 - accuracy: 0.8882 - 18s/epoch - 10ms/step
Epoch 3/5
1875/1875 - 17s - loss: 0.2538 - accuracy: 0.9054 - 17s/epoch - 9ms/step
Epoch 4/5
1875/1875 - 17s - loss: 0.2224 - accuracy: 0.9169 - 17s/epoch - 9ms/step
Epoch 5/5
1875/1875 - 16s - loss: 0.1948 - accuracy: 0.9266 - 16s/epoch - 9ms/step
313/313 - 2s - loss: 0.2915 - accuracy: 0.8905 - 2s/epoch - 5ms/step
...
Epoch 5/5
1875/1875 - 17s - loss: 0.2707 - accuracy: 0.9013 - 17s/epoch - 9ms/step
313/313 - 2s - loss: 0.2654 - accuracy: 0.8988 - 2s/epoch - 5ms/step
Model with Dropout Test Accuracy: 0.8988
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Lab Experiment - 5

5. Design and implement an image classification model to classify a dataset of images using deep feed-forward neural network. Record the accuracy corresponding to the number of epochs. Use MNIST dataset.

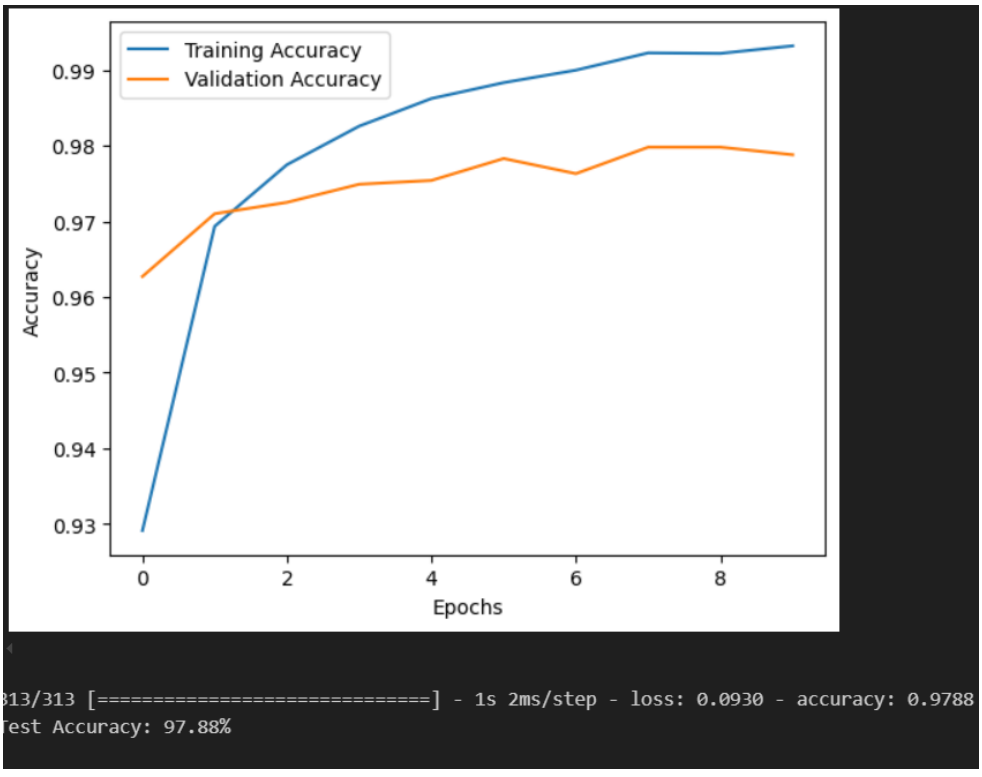
Program:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt


# Load and preprocess the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train.reshape(-1, 784) / 255.0, x_test.reshape(-1, 784) / 255.0
# Define and compile the model
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),  # First hidden layer
    Dense(64, activation='relu'),  # Second hidden layer
    Dense(10, activation='softmax')  # Output layer with 10 neurons (for 10 classes)
])
# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# Train the model
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=10,
batch_size=32)
# Plot accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
```

```
plt.legend()
plt.show()

# Evaluate the model
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

Output:

```
Epoch 1/10
WARNING:tensorflow:From c:\Users\Shreya\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedT

WARNING:tensorflow:From c:\Users\Shreya\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.execu

1875/1875 [==============================] - 12s 4ms/step - loss: 0.2388 - accuracy: 0.9291 - val_loss: 0.1225 - val_accuracy: 0.9627
Epoch 2/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.1010 - accuracy: 0.9693 - val_loss: 0.0919 - val_accuracy: 0.9710
Epoch 3/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0727 - accuracy: 0.9775 - val_loss: 0.0871 - val_accuracy: 0.9725
Epoch 4/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0537 - accuracy: 0.9826 - val_loss: 0.0825 - val_accuracy: 0.9749
Epoch 5/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0429 - accuracy: 0.9862 - val_loss: 0.0839 - val_accuracy: 0.9754
Epoch 6/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0352 - accuracy: 0.9883 - val_loss: 0.0830 - val_accuracy: 0.9783
Epoch 7/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0301 - accuracy: 0.9900 - val_loss: 0.0901 - val_accuracy: 0.9763
Epoch 8/10
...
Epoch 9/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0230 - accuracy: 0.9922 - val_loss: 0.0869 - val_accuracy: 0.9798
Epoch 10/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.0204 - accuracy: 0.9932 - val_loss: 0.0930 - val_accuracy: 0.9788
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



Lab Experiment - 6

6. Implement Bi-directional LSTM for Sentiment analysis on movie reviews.

Program:

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense

# Parameters
max_features = 10000  # Top 10000 most common words
max_len = 200  # Max length of each review

# Load and preprocess data
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
x_train, x_test = map(lambda x: pad_sequences(x, maxlen=max_len), (x_train, x_test))

# Build, compile, and train the model
model = Sequential([
    Embedding(input_dim=max_features, output_dim=64, input_length=max_len),
    Bidirectional(LSTM(32)),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.2)

# Evaluate the model
print(f"Test Accuracy: {model.evaluate(x_test, y_test)[1]:.2f}")
```

```
# Test on a custom review
example_review = "The movie was absolutely amazing, I loved it!"
encoded_review = [imdb.get_word_index().get(word, 2) for word in
example_review.lower().split()]
padded_review = pad_sequences([encoded_review], maxlen=max_len)
prediction = model.predict(padded_review)[0][0]
print(f"{'Positive' if prediction < 0.5 else 'Negative'} sentiment with confidence {1 -
prediction if prediction < 0.5 else prediction:.2f}")
```

Output:

Lab Experiment - 7

7. Implement the standard VGG16 and 19 CNN architecture model to classify multicategory image dataset and check the accuracy.

Program:

A.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load and preprocess Fashion MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
x_train, x_test = x_train[..., None] / 255.0, x_test[..., None] / 255.0  # Normalize and add channel
y_train, y_test = to_categorical(y_train, 10), to_categorical(y_test, 10)

# Define a simpler VGG-like model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

# Compile and train the model
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5, batch_size=128)

# Evaluate the model
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy:.2f}")
```

B.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load and preprocess Fashion MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
x_train, x_test = x_train[..., None] / 255.0, x_test[..., None] / 255.0  # Normalize and add channel
y_train, y_test = to_categorical(y_train, 10), to_categorical(y_test, 10)

# Define a VGG19-inspired model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
```

```
    Dense(10, activation='softmax')
])

# Compile and train the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=5, batch_size=128)

# Evaluate the model
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy:.2f}")
```

Output:

A.

```
Epoch 1/5
WARNING:tensorflow:From c:\Users\Shreya\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\utils\tf_utils.py:492: The n

WARNING:tensorflow:From c:\Users\Shreya\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\engine\base_layer_utils.py:3

469/469 [==============================] - 15s 29ms/step - loss: 0.6655 - accuracy: 0.7617 - val_loss: 0.4122 - val_accuracy: 0.8488
Epoch 2/5
469/469 [==============================] - 14s 30ms/step - loss: 0.4209 - accuracy: 0.8484 - val_loss: 0.3673 - val_accuracy: 0.8628
Epoch 3/5
469/469 [==============================] - 15s 33ms/step - loss: 0.3661 - accuracy: 0.8687 - val_loss: 0.3174 - val_accuracy: 0.8837
Epoch 4/5
469/469 [==============================] - 14s 31ms/step - loss: 0.3348 - accuracy: 0.8778 - val_loss: 0.3085 - val_accuracy: 0.8865
Epoch 5/5
469/469 [==============================] - 15s 31ms/step - loss: 0.3091 - accuracy: 0.8891 - val_loss: 0.2937 - val_accuracy: 0.8909
313/313 [==============================] - 2s 6ms/step - loss: 0.2937 - accuracy: 0.8909
Test Accuracy: 0.89
```

B.

```
Epoch 1/5
469/469 [==============================] - 36s 73ms/step - loss: 0.6677 - accuracy: 0.7545 - val_loss: 0.4258 - val_accuracy: 0.8427
Epoch 2/5
469/469 [==============================] - 29s 62ms/step - loss: 0.4056 - accuracy: 0.8537 - val_loss: 0.3614 - val_accuracy: 0.8703
Epoch 3/5
469/469 [==============================] - 29s 62ms/step - loss: 0.3393 - accuracy: 0.8781 - val_loss: 0.3028 - val_accuracy: 0.8889
Epoch 4/5
469/469 [==============================] - 29s 62ms/step - loss: 0.2973 - accuracy: 0.8926 - val_loss: 0.2738 - val_accuracy: 0.9000
Epoch 5/5
469/469 [==============================] - 28s 60ms/step - loss: 0.2755 - accuracy: 0.9012 - val_loss: 0.2621 - val_accuracy: 0.9044
313/313 [==============================] - 2s 8ms/step - loss: 0.2621 - accuracy: 0.9044
Test Accuracy: 0.90
```